

Evaluation of the Code Generated By Large Language Models: The State of the Art

Zhihao Ying, Dave Towey*, Yifan Zhang

*School of Computer Science, University of Nottingham Ningbo China,
Zhejiang 315100, People's Republic of China
{Zhihao.Ying2, Dave.Towey, Yifan.Zhang}@nottingham.edu.cn*

Abstract—The rapid development of Large Language Models (LLMs), such as ChatGPT and DeepSeek, has revolutionized software development, particularly in the domain of automated code generation. These models, built on architectures like the Transformer, have demonstrated remarkable capabilities in generating human-like text and source code, significantly enhancing developer productivity and reducing development time. However, the widespread adoption of LLMs for code generation raises concerns regarding the reliability, quality, and potential risks associated with the generated code. This article illustrates and analyzes the state of the art in evaluating LLM-generated code, summarizing research findings, and application areas. **This paper highlights the challenges in distinguishing between machine-generated and human-written code, as well as the potential for LLMs to introduce security vulnerabilities and maintainability issues.** We discuss the implications of these findings for both researchers and practitioners, emphasizing the need for continued research in the evaluation of LLM-generated code. Finally, we identify gaps in the literature and propose future research directions, such as the development of more robust benchmarks and improved evaluation metrics. By providing a thorough overview of the current landscape, this paper provides a valuable resource for researchers and practitioners interested in LLM's code generation capabilities and limitations. We also highlight the importance of ongoing evaluation and refinement of these models to ensure their safe and effective integration into software-development practices.

Index Terms—Artificial intelligence; large language model; code generation; code evaluation; code error;

I. INTRODUCTION

The advent of Large Language Models (LLMs) has brought about a paradigm shift in software development, particularly in the domain of automated code generation. These models possess the ability to understand and generate human-like text, approaching human-level performance, with ChatGPT [1]–[3] (from OpenAI¹) and DeepSeek² [4]–[12] being two typical examples. Both of them have exhibited outstanding performance in various daily activities, especially in code-related tasks, and have become essential tools for individuals, academia, and industry, significantly enhancing productivity across multiple domains. Among the many applications, LLMs have shown exceptional promise in generating source code based on natural language descriptions, significantly enhancing developer productivity and reducing development time.

¹<https://openai.com/> (accessed: 1 February 2025)

²<https://www.deepseek.com/> (accessed: 1 February 2025)

Automated code generation refers to the process of producing source or executable code from given specifications or requirements [13]. This technology can offer a lot of benefits, including the ability to automate repetitive coding tasks, reduce errors, and allow developers to focus on higher-level design and logic. There are a number of techniques that can be used to support the process of automated code generation, such as LLMs [14]. However, the widespread adoption of LLMs for automated code generation also raises concerns regarding the reliability, quality, and potential risks associated with the generated code. As these models become increasingly integrated into software-development workflows, there is a great need to evaluate and understand their performance, limitations and challenges.

This paper describes and analyzes the current state-of-the-art performance of LLM-generated code. Specifically, this paper summarizes the research findings and application areas of LLM-generated code, while also identifying potential future research directions. **The key research questions of this paper include the types of errors observed in LLM-generated code, the categorization of these errors, and the error types that emerge from these categorizations.** By providing a thorough overview of the current landscape, this paper provides valuable resource for researchers and practitioners interested in the capabilities and limitations of LLMs in code generation. It also highlight the importance of ongoing evaluation and refinement of these models to ensure their safe and effective integration into software-development practices.

In Section II, we briefly introduce the concepts of LLMs and their applications in code generation. Section III presents a structured review methodology, detailing the motivation, research questions, source material, and search strategy used. The review presented in Section IV lists the multiple errors and various error types identified in LLM-generated code, such as compilation errors, runtime errors, time-limit-exceeded errors, and issues related to code style, maintainability, and performance. This paper also explores the methodologies used to evaluate the quality of LLM-generated code. Finally, Section V concludes the paper by summarizing the key findings and discussing potential research directions.

II. LARGE LANGUAGE MODELS (LLMs)

The development and use of LLMs, such as ChatGPT and DeepSeek, for code generation have gained a lot of attention in recent years, supported by the large number of open-source code-bases and the need to improve developer efficiency. These models have shown potential in automating assessments, providing feedback, and modeling complex behaviors. LLMs have demonstrated state-of-the-art performance on various code-related tasks, including code generation [15]–[23], program repair [24], education [25]–[28], automated testing [29], [30] and code summarization [31]. In particular, prominent LLMs including CODEX [23], CodeGen [32], INCODER [33], and PolyCoder [34] have been developed and extensively evaluated for code generation, where the model generates code snippets based on natural language descriptions, such as docstrings, of the desired functionality.

A. OpenAI and ChatGPT

OpenAI has developed an LLM called ChatGPT, which interacts in a conversational manner. The dialogue format enables it to answer follow-up questions, admit errors, question incorrect premises, and refuse to answer improper requests. Until now, the OpenAI team has released multiple versions of ChatGPT [1]–[3]: The currently available models include OpenAI o3-mini, OpenAI o1, OpenAI o1-mini, GPT-4o, and GPT-4o mini. OpenAI used Reinforcement Learning from Human Feedback [35], [36] to train and develop ChatGPT. Specifically, the OpenAI team trained an initial model using supervised fine-tuning: A human AI trainer provided a dialogue in which they acted as both parties—the user and the AI assistant. They then provided written suggestions for trainers on models to assist them in writing responses, and converted this new dialogue dataset into a dialogue format. The OpenAI team conducted a dialogue between an AI trainer and a ChatBot to collect experimental data, comparing responses from two or more models ranked by quality. Then they randomly selected a message written by a model, sampled several alternative completion methods, and had an AI trainer rank them. Using these models, the OpenAI team fine-tuned the model and iterated this process multiple times. However, ChatGPT also has some limitations [1]–[3], listed as follows:

- It may sometimes provide answers that sound reasonable but are incorrect.
- It is easily influenced by adjusting input phrases or trying one prompt many times.
- It is often too verbose and excessively uses certain phrases. For example, it may reiterate that it is an LLM developed by OpenAI.
- When users ask ambiguous requests, it is currently unable to raise clarification questions; instead, it typically guesses the user's intention.
- It is currently unable to refuse to answer improper requests.

OpenAI has also trained a GPT-4-based AI model named CriticGPT [37] to identify errors in the code generated by

language models. This model is specifically designed to catch bugs in the code generated by ChatGPT, acting as an AI assistant to enhance human oversight of AI systems and improve the alignment between AI behavior and human expectations. The development of CriticGPT uses a technique called Reinforcement Learning from Human Feedback [35], [36], assisting human reviewers in making the outputs of LLMs more accurate. The researchers trained CriticGPT on a dataset of code samples with intentionally inserted bugs, allowing it to learn how to identify and flag various coding errors. The results of the study showed that the annotations given by CriticGPT are more preferred than manual annotations.

B. Other Popular LLMs

In addition to ChatGPT, many other LLMs have been developed, such as DeepSeek, Claude, Grok, and Gemini. For instance, DeepSeek has also emerged as a pivotal player in the AI landscape, particularly in the development of LLMs and their applications. Its research spans multiple domains, including natural language processing, code intelligence, and mathematical reasoning. Its available versions include DeepSeek R1 [12], DeepSeek V3 [10], DeepSeek Coder V2 [8], DeepSeek VL [5], DeepSeek V2 [9], DeepSeek Coder [7], DeepSeek Math [4], and DeepSeek LLM [11]. DeepSeek's technologies have been widely adopted in both academic and industrial settings. For instance, its code intelligence models, such as DeepSeek Coder [7] and DeepSeek Coder V2 [8], have set new benchmarks in programming tasks, while its reasoning models have been integrated into educational tools and scientific research platforms.

One of the most recent version published by DeepSeek-AI team is the DeepSeek-V3 [10], which can balance model performance and computational cost. One of the core advantage of DeepSeek-V3, for instance, is its efficient mixture-of-experts architecture. By activating only a subset of experts during inference, DeepSeek-V3 achieves state-of-the-art results while significantly reducing resource consumption. This model has demonstrated exceptional capabilities across multiple complex tasks while significantly reducing operational costs, offering novel insights into the practical application of large-scale models. Through efficient inference and cost-effective training methodologies, DeepSeek-V3 aims to push the boundaries of open-source model capabilities while competing in performance with closed-source models such as GPT-4o [3] and Claude-3.5 Sonnet (A popular LLM that was developed by Anthropic³ and was released on 20 June 2024). For instance, this architecture has been validated across various benchmarks and outperforms many closed-source models [10]–[12].

Additionally, Claude, Gemini and Grok are also state-of-the-art LLMs that were rated as the top-performing models [38]. Claude is a very popular LLM and is the first hybrid reasoning model on the market, which is developed by Anthropic, and its most recently-released version is Claude-7 Sonnet. Gemini⁴

³<https://www.anthropic.com/> (accessed: 1 February 2025)

⁴<https://deepmind.google/technologies/gemini/> (accessed: 1 February 2025)

is a powerful LLM released by Google, and its most recent version is Gemini 2.0, including Gemini 2.0 Pro, Gemini 2.0 Flash, Gemini 2.0 Flash Thinking, and Gemini 2.0 Flash-Lite. Grok is also a competitive player in the AI space, which is developed and released by xAI⁵, whose newest version is Grok 3.0.

III. REVIEW METHOD

To analyze the current state of the art in the evaluation of LLM-generated code, we employed a structured and systematic approach, guided by Kitchenham and Charters [39] and Petersen et al. [40], to ensure a comprehensive and unbiased analysis of existing studies. In addition, the literature review was an iterative process with multiple rounds of searching, data extraction, and analysis. This iterative approach allowed us to refine our research questions, update our search strategy, and ensure that the review remained comprehensive and up-to-date. Below, we outline the methodology used to conduct the literature review.

A. Motivation and Research Question (RQ)

The scope of this review is focused on the evaluation of code generated by LLMs, particularly in the context of software development: We selected the studies that explore the reliability, quality, and potential risks associated with LLM-generated code. More specifically, we investigated, structured and classified the identified previously identified code errors in the LLM-generated code. In order to achieve this aim, we designed the following four Research Questions (RQs):

- RQ1: What methodologies are used to evaluate the quality of LLM-generated code?
- RQ2: What types of errors have been identified in LLM-generated code?
- RQ3: How have these errors been categorized in existing studies?
- RQ4: What error types emerge from the categorization of errors in LLM-generated code?

We proposed RQ1 and RQ2 to obtain an in-depth view on the coding performance of LLMs: Can an LLM produce correct code under different kinds of situations? If not, what type of errors may exist? The answer to RQ3 will provide an overview of the classification of the identified errors, and the answer to RQ4 will summarize the classification details. These questions help structure the review and ensure that the analysis is focused on key aspects of LLM-generated code evaluation.

B. Source Material and Search Strategy

1) *Databases and Sources:* We conducted a thorough search of academic databases, including IEEE Xplore, ACM Digital Library, SpringerLink, and Google Scholar. Additionally, we reviewed pre-prints from arXiv and other relevant repositories to ensure that the latest research was included.

⁵<https://x.ai/> (accessed: 1 February 2025)

2) *Keywords and Search Terms:* The search terms used included combinations of the following keywords: Large Language Models, LLM-generated code, code evaluation, ChatGPT, DeepSeek, code quality, code errors, and software development.

3) Inclusion and Exclusion Criteria:

- Inclusion: Studies that focus on the evaluation of LLM-generated code, including empirical studies, case studies, and benchmark evaluations. Both peer-reviewed articles and pre-prints were considered. In addition, due to the rapid development and iteration of LLMs, only the papers published within the last three years were included in this paper.
- Exclusion: Studies that do not directly address LLM-generated code or those that focus solely on theoretical aspects without empirical evaluation.

C. Coding Benchmark

LLM-based code synthesis is largely evaluated based on functional correctness, which is typically assessed by running test cases to check the desired outputs. HUMANEVAL [23] is one of the popular and most widely studied human-written benchmarks for LLM-based code synthesis, consisting of 164 pairs of Python function signatures with doc-strings and the associated test cases for correctness checking. Additionally, each HUMANEVAL problem is equipped with a reference solution. Another Python-focused dataset, MBPP [41], was created by crowd-sourcing participants to write a total of 974 programming problems, each comprising a problem statement (e.g., doc-string), the function signature, and three test cases. Beyond Python, there are other benchmarks targeting additional languages such as Spider (e.g., SQL) [42], HUMANEVAL-X (e.g., C++, JavaScript, and Go) [43], CodeContests (e.g., C++ and Java) [44], and MultiPL-E (e.g., extending HUMANEVAL and MBPP to 18 programming languages) [45]. More recently, Jimenez et al. [46] developed a more realistic code-synthesis benchmark by collecting GitHub issues along with the corresponding code base and tests to evaluate the capability of LLMs in performing real-world software-engineering tasks.

D. A LLM-Generated Code Evaluation Framework

In this section, we propose a three-layer framework to systematically assess the performance of LLM in generating code and the quality of the generated code:

1) Functional Correctness:

- Compilation errors: The generated code fails to compile. Common errors include the use of undeclared functions, the use of wrong function declarations, and the redefinition of main functions.
- Runtime errors: The code crashes during execution. Common errors include heap-buffer-overflow, integer overflow, out-of-bound, type error, and null pointer dereference.
- Logical errors: Incorrect logic is implemented in the code and the code produces unexpected output. Common errors include edge case failures.

- Syntax errors: The code violates the rules of the programming language. Common errors include missing semicolons, brackets, and wrong indentation of the code.

2) Code Quality:

- Reliability: The system can still function normally in adversity, which refers to a series of errors such as hardware failure, software failure, human error, and so on. "Function normally" means that the system can correctly complete its functions and achieve the expected performance level.
- Maintainability (e.g., style, documentation): (1) Operability: It represents the ability of the code to run smoothly under normal circumstances; (2) Lucidity: It represents the simplicity of the code; (3) Modifiability: It represents the capability of the code to successfully integrate new and unforeseen features.
- Security (e.g., hard-coded credentials): The security level of the code.
- Performance (e.g., time/space complexity): Common errors include time-limit-exceeded error, which represent that the code takes too long to run and execution time exceeds the given time in any coding contest.

3) Context Adaptability:

- Domain knowledge integration: The ability of generated code to meet the actual needs of specific application scenarios.
- Cross-language generalization: The ability of LLMs to adapt to the syntax and features of different programming languages when generating or understanding code. Specifically, it measures whether the model can generate high-quality code in multiple programming languages such as Python, Java, C++, Rust, and so on, rather than being limited to a single language.

IV. STATE OF THE ART IN LLM-GENERATED CODE EVALUATION

A. Investigation of LLMs' Code Generation Performance

Mitchell et al. [47] introduced DetectGPT, a zero-shot learning [48], [49] approach for checking and detecting whether a piece of text is machine-generated or not. DetectGPT uses random perturbations of a candidate passage (generated by a pre-trained model) to determine if the given text was generated by a specific LLM. The method does not require training a separate classifier or collecting datasets of real text.

However, DetectGPT was not developed specifically for code evaluation. Considering this, DetectCodeGPT was proposed by Shi et al. [50], which aims to analyze the differences between machine-generated and human-written code in terms of lexical diversity, conciseness, and naturalness, and to propose a novel method, DetectCodeGPT, for detecting machine-generated code. DetectCodeGPT perturbs code by strategically

inserting spaces and newlines, leveraging the observed differences in stylistic tokens between machine-generated and human-written code. This approach eliminates the need for external pre-trained models, enhancing both efficiency and effectiveness. Shi et al. [50] evaluated DetectCodeGPT on two datasets (CodeSearchNet and The Stack) across six code-generation models.

B. Errors Identified in LLM-Generated Code

Liu et al. [15] collected and used 728 algorithm problems from LeetCode platform⁶, which is an online platform designed for coding practice: It includes more than 1,500 coding problems spanning multiple topics such as data structures, algorithms, and dynamic programming. They then manually examined the source-code snippets generated by LLMs. In total, five programming languages were selected, including C, C++, Java, python3, and JavaScript, and the following errors were identified.

- Compile Error: In addition to the common compile errors, seven less common errors were also identified, including uninitialized variables, wrong methods, undeclared variables, invalid operators, error of *include*, redefinitions, and incompatible parameter types.
- Runtime Error: In addition to the common runtime errors, a total of nine less common errors were also identified, including type error, null pointer dereference, wrong method name, uninitialized variables, value error, heap-use after-free, recursion error, divided by zero, and undefined-behavior.
- Time-Limit-Exceeded Error: Common causes include use of inefficient algorithms (e.g., brute-force search) and complex logic leading to high time complexity.
- Other common errors: Additionally, Liu et al. [15] also manually inspect the code snippets and identify common errors related to code complexity and security. The differences in complexity levels (such as built-in libraries, different algorithms, and implementation of logic) and security levels were also explored and summarized.

Liu et al. [15] manually analyzed source-code snippets and categorized the causes of errors (code with wrong answers) from a logical perspective into the following categories:

- Runtime Errors Types:
 - Wrong Detail: Errors in certain details, such as misunderstandings of the problem or inconsistencies between the code logic and the problem description.
 - Misunderstanding Certain Content: The code uses an appropriate algorithm but fails to meet the core conditions of the problem.
 - Misunderstanding Problem: ChatGPT completely misunderstands the problem description and uses the wrong algorithm.

In addition to the runtime error types, Liu et al. [15] also reported the possible causes of compile errors as follows.

⁶<https://leetcode.com/> (1 February 2025)

- Compile Errors Types:
 - Errors in Languages: Errors arising from the properties of the language used, such as C and C++.
 - Errors between Languages: Errors arising from the similarity between languages like C and C++.

Finally, through manual analysis, Liu et al. [15] presented the final kind of errors, which include the following three detailed errors.

- Time-Limit-Exceeded Error Types:
 - Aligned but Inefficient Algorithm Implementation: The algorithm used in the ChatGPT-generated code is aligned with the requirement given in the question, but some parts are not efficient.
 - Functionally Correct but Misaligned Algorithm: The algorithm used by ChatGPT is functionally correct for the given question but is misaligned — it is inefficient.
 - Functionally Incorrect Algorithm: The algorithm used by ChatGPT is functionally incorrect for the given question, which is also slow (due to the incorrect function) to resolve the given question, such as test cases. The algorithm may be aligned or misaligned, such as a brute-force method.

Sarsa et al. [51] explored methods for systematically evaluating the quality of LLM-generated codes, relevant for distinguishing between LLMs and human-generated codes. They also identified types of errors in LLM-generated codes.

- Error Types Identified in LLM-Generated Programming Exercise:
 - Sensibility: Whether the generated programming exercise makes sense as a learning task, based on typical problem-solving structures in programming education. For example, whether it aligns with real-world problems and educational contexts.
 - Novelty: Whether the generated exercise is unique, ensuring that it does not merely replicate common problems found elsewhere, thus promoting creativity in generated content.
 - Readiness: Whether the generated exercise is immediately usable in educational contexts. For instance, it includes well-defined solutions, test cases, and problem descriptions that do not require substantial modifications.
- Evaluation Criteria for Programming Exercises:
 - Correctness: Whether the generated code in the sample solutions works as intended, passes the provided test cases, and aligns with the task description.
 - Test Coverage: The extent to which the test cases are able to validate the solution and ensure its correctness. This includes checking for edge cases and corner cases that could break the solution.
 - Code Quality: The syntactical and logical correctness of the code, ensuring that it adheres to best coding practices and is free from basic errors or inconsistencies.

In addition to code evaluation, Sarsa et al. [51] also proposed several metrics for the evaluation of LLM-generated code explanations.

- Error Types Identified in LLM-Generated Code Explanations:
 - Coverage: The proportion of the code that is explained by the LLMs, with a focus on ensuring every line of code is covered in a clear and meaningful way.
 - Accuracy: The correctness of the explanation, specifically whether the LLMs accurately describes the logic of the code, including explanations of variables, conditionals, loops, and other key programming constructs.
 - Clarity and Structure: How clearly the explanation is written, ensuring that the explanation is easy to understand for students, with the correct level of detail provided.
- Evaluation Criteria for Code Explanations:
 - Clarity: Whether the explanations are easy to follow and written in a way that matches the cognitive level of the target audience (beginner or intermediate programmers).
 - Accuracy: The correctness of the explanation, which includes ensuring that all logical elements in the code are correctly interpreted and explained.
 - Consistency: The uniformity in how explanations are generated for different parts of the code, ensuring a cohesive and coherent explanation style throughout.

Guo et al. [52] reported on a case study deeply analyzing and investigating the cause of the lower performance of CodeReviewer (a pre-trained model for reviewing source codes that make use of four specific pre-training tasks) [53] compared to ChatGPT, presenting a total of four major causes:

- Inaccurate understanding of the review content: Through manual analysis, they presented that some code reviews contain unclear information, such as ambiguous location references, unclear changes, or requiring viewer models to comprehend.
- Over deletion: CodeReviewer model exhibits a tendency to inaccurately delete code snippets.
- Extra modification: Sometimes CodeReviewer model may introduce unnecessary modifications to code snippets that do not require any changes.
- Hard to understand the ground truth provided in the code block: Through manual analysis, they reported that, in some cases, reviewers have accurately suggested changes within the code block. However, CodeReviewer fails to recognize that the code within these blocks represents the ground truth, leading to incorrect modifications.

Then, through manual analysis, they presented two major categories of root causes for the errors found in their work:

- Inaccurate Measurement Category: Refers to false positives where the predicted refinement by ChatGPT is correct based on manual inspection, but the measurement metrics are low due to strict matching. A total of four

kinds of root causes were identified and added to this category:

- Insignificant Omission: ChatGPT failed to return unmodified code segments but correctly returned the modified parts.
- Unexpected Grammar Fix: ChatGPT fixed grammar errors in the documentation that were not present in the ground truth revised code.
- Code Style Difference: The predicted code given by ChatGPT is semantically identical to the ground truth revised code, with differences only in whitespace, line breaks, and other code style aspects that do not affect code semantics, and the review comment did not explicitly prohibit the change of code style.
- Reasonable Improvement: ChatGPT's modifications are highly reasonable and represent an improvement over the original version.
- Incorrect Prediction Category: Refers to true positive cases where ChatGPT made incorrect answers compared to the ground truth revised code. In total, three types of root causes were identified and added to this category:
 - Need domain knowledge: The review comment does not provide the necessary repository-related domain knowledge to complete the modification.
 - Unclear location: The review comment does not provide a specific location for the code to be modified.
 - Unclear changes: The review comment has a lower information level, causing ChatGPT to be unable to determine the specific modifications needed, resulting in under-performance.

Siddiq et al. [16] explored and analyzed the quality and influence issues of ChatGPT-generated code. They chose two programming languages, Python and Java, and selected the DevGPT dataset [54] to explore the code and security issues in the ChatGPT-generated codes using three static analyzers: Bandit [55], CodeQL [56] and Pylint [57]. They also explored whether the code generated by ChatGPT can be merged into the code base.

- Java code: The problems include not using proper JavaDoc in public methods, constructors, or variables, dead code, unused Maven dependencies, and unused and unread local variables. In terms of security smells, ChatGPT mainly provides hard-coded credentials in the code. Hard-coded credential, also often referred to as embedded credentials, is a software development practice in which authentication data (user IDs and passwords) are embedded directly into the source codes of an executable program.
- Python code: Most messages for Python files are error and convention types. The top messages for each category are undefined variables, missing doc-strings, too few public methods in a generated class, and redefining a variable declared in the outer scope. The codes generated by ChatGPT may also suffer from security issues. The top three security issues are requests without a timeout, using

pseudo-random generators, and using asserts.

According to a thorough manual analysis of the ChatGPT-generated code, Liu et al. [20] presented four different categories of errors: Compilation and runtime error, wrong output error, code style and maintainability error, and performance and efficiency error. Among them, the wrong outputs error and the code style and maintainability error are the most common problems faced by ChatGPT in generating codes, while the compilation and runtime error and the performance and efficiency error are relatively less common. Additionally, they also investigated and listed the top 10 common code style and maintainability errors and the compilation and runtime errors, shown in Table I.

- Compilation and Runtime Error: Compilation and runtime errors encompass issues that prevent the correct execution of the code. These errors can arise due to various factors, such as incorrect use of a programming language's syntax, improper handling of data structures, invalid input, or exceeding the bounds of an array. The errors often lead to failures during compilation or runtime, and they need to be resolved before the program can function as intended.
- Wrong Output Error: Wrong outputs represent issues in the code that cause it to produce incorrect results or fail to meet the problem requirements. These errors can stem from incorrect algorithms, improper handling of edge cases, or other inaccuracies in the desired logic. These errors can occur even when the code is syntactically correct and free from any runtime errors.
- Code Style and Maintainability Error: This category encompasses issues related to the code's formatting, organization, and adherence to established style guides. Poorly styled or organized code is difficult to read, understand, and maintain. Examples include inconsistent indentation, overly long lines, or lack of meaningful variable names.
- Performance and Efficiency Error: Performance and efficiency issues arise when the code is not optimized for execution speed or resource usage. These issues can lead to slow or resource-intensive programs, which can negatively impact the user experience or system stability. Examples include inefficient algorithms, unnecessary memory allocations, or redundant calculations.

Feng et al. [22] sent the ChatGPT-generated code (in Python) to Flake8 as individual python code files to check for quality issues and errors, and identified the codes that cause errors for each individual file, as well as their codes, descriptions, and positions, which are summarized as follows (sorted by frequency of occurrence):

- E501: Line too long (114 >79 characters).
- E231: Missing whitespace after ",",
- E302: Expected 2 blank lines, found 1.
- W293: Blank line contains whitespace.
- E402: Module level import not at top of file.
- E305: Expected 2 blank lines after class, found 1.
- E265: Block comment should start with "#".

TABLE I
THE LIST OF IDENTIFIED ERRORS AND CORRESPONDING DESCRIPTIONS [20]

Errors	Programming languages	Categories	Descriptions
Top 10 Compilation and Runtime Errors	Java and Python	Division by Zero	Attempt to divide by zero
		Illegal Index	Accessing an array or list with an invalid index
		Concurrent Modification	Modifying a collection during iteration
		Empty Collection Access	Accessing an element from an empty collection
		Key Not Found	Accessing a non-existent key in a dictionary or map
		Null Reference	Attempt to access an attribute or method of a null object
		Type Mismatch	Using an incorrect data type in an operation or function call
		Resource Limit Exceeded	Exceeding the system's resource limits
		Syntax Error	Incorrect syntax or structure in the code
		Undefined Variable	Accessing or using a variable that has not been defined
		Attribute Not Found	Attempt to access a non-existent attribute or method of an object
		Duplicate Variable	Defining a variable more than once in a scope
Top 10 Code Style and Maintainability Errors	Python	Consider Using Enumerate	Used when code that iterates with range and len is encountered.
		No Else Return	Used in order to highlight an unnecessary block of code following an if containing a return statement.
		Unused Variable	Used when a variable is defined but might not be used.
		Redefined Built in	Used when a variable or function overrides a built-in.
		Consider Using Dict Items	Used when iterating over the keys of a dictionary and accessing the value by index look up.
		Avoid Ambiguous Names	Used when code use variables named "I", "O", or "l".
		Too Many Branches	Used when a function or method has too many branches, making it hard to follow.
		Too Many Locals	Used when a function or method has too many local variables.
		Blank Lines	Nested functions should contain 1 blank line between their definitions.
		Inconsistent Return Statements	Either all return statements in a function should return an expression, or none of them should.
	Java	Multiple Variable Declarations	Each variable declaration must be in its own statement.
		Avoid Reassigning Parameters	Emitted when incoming parameters are reassigned values.
		For Loop Can Be For each	Used to recommend to use for each loop instead of loop.
		Redundant Modifier	Emitted when a modifier is redundant.
		Right Curly	Emitted when right curly in a code violates common conventions.
		Visibility Modifier	Used to recommend that a variable should not be public.
		N Path Complexity	Used when a method has too many acyclic execution paths.
		Loose Coupling	Used when using implementation types instead of interface.
		Hidden Field	Emitted when a local variable or a parameter does not shadow a field that is defined in the same class.
		Use Concurrent HashMap	Recommend to use the Concurrent HashMap implementation.

- W291: Trailing whitespace.
- E999: Syntax Error (invalid syntax).
- E227: Missing whitespace around bit-wise or shift operator.
- W292: No newline at end of file.
- E101: Indentation contains mixed spaces and tabs.
- F401: "torch" imported but unused.
- W191: Indentation contains tabs.
- W391: Blank line at end of file.
- E261: At least two spaces before inline comment.
- E225: Missing whitespace around operator.
- F811: Redefinition of unused "pymesh" from line 5.
- E902: TokenError: EOF in multi-line statement.
- F821: Undefined name "output value".
- E741: Ambiguous variable name "I".

C. Limitations of Current Benchmarks

Current benchmarks exhibit two biases:

- 1) Language Bias: Most of the tasks focused on popular programming languages only (e.g., C++, Java and Python) but neglecting other languages such as Rust.
- 2) LLM-Generated Code Evaluation Method Bias: Many experiments only focus on one or two different evaluation method, without a unified code quality verification framework. This makes it difficult to compare different research results with each other. Considering this, we propose a three-layer framework to systematically evaluate the performance of LLM in generating code and the quality of the generated code in Section III-D.

V. CONCLUSION

In this paper, we have examined the state of the art in evaluating code generated by LLMs, with a particular focus

on models like ChatGPT. The rapid advancement of LLMs has significantly impacted software development, offering improvements in both productivity and efficiency. However, the widespread adoption of these models also raises concerns regarding the reliability, quality, and potential risks associated with their generated code.

Through a systematic review of existing literature, we identified various errors and error types in LLM-generated code, including compilation errors, runtime errors, time-limit-exceeded errors, and issues related to code style, maintainability, and performance. We also explored the methodologies used to evaluate the quality of LLM-generated code, such as functional correctness, test coverage, and code quality metrics. Additionally, we discussed the challenges in distinguishing between machine-generated and human-written code, as well as the potential for LLMs to introduce security vulnerabilities and maintainability issues.

The findings from this paper highlight the need for continued research and development in the evaluation of LLM-generated code. Future work should focus on developing more robust benchmarks, improving the interpretability of LLM outputs, and addressing the ethical and security implications of using LLMs in software development. Furthermore, there is a need for more comprehensive studies that explore the long-term impact of LLM-generated code on software maintenance and evolution. In this article, we propose a code evaluation framework, but it can still be improved. Therefore, our future work includes continuing to improve the framework we proposed and its empirical applications. In addition, more details can be integrated into this framework to guide the specific implementation of code evaluation, which remains an open field for future research and development in the field of code

evaluation.

In conclusion, while LLMs like ChatGPT have demonstrated well performance in code generation, their integration into software-development workflows must be approached with caution. By addressing the identified challenges and leveraging the strengths of LLMs, we can harness their potential to revolutionize software development while mitigating associated risks.

DECLARATION OF COMPETING INTEREST

The authors declare that there is no conflict of interest or personal relationships that could have appeared to influence the work reported in this paper.

ACKNOWLEDGMENTS

The authors also acknowledge the support from the University of Nottingham Ningbo China (UNNC), Artificial Intelligence and Optimization (AIOP) research group and the Faculty of Science and Engineering (FoSE).

REFERENCES

- [1] OpenAI. gpt-4 technical report, 2023. <https://openai.com/index/gpt-4-research/>.
- [2] OpenAI. gpt-4v(ision) system card, 2024. <https://openai.com/index/gpt-4v-system-card/>.
- [3] OpenAI. Hello gpt-4o, 2024. <https://openai.com/index/hello-gpt-4o/>.
- [4] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. arXiv:abs/2402.03300.
- [5] Haoyu Lu, Wen Liu, Bo Zhang, Bingxuan Wang, Kai Dong, Bo Liu, Jingxiang Sun, Tongzheng Ren, Zhuoshu Li, Hao Yang, Yaofeng Sun, Chengqi Deng, Hanwei Xu, Zhenda Xie, and Chong Ruan. Deepseek-vl: Towards real-world vision-language understanding, 2024. arXiv:abs/2403.05525.
- [6] Zhiyu Wu, Xiaokang Chen, Zizheng Pan, Xingchao Liu, Wen Liu, Damai Dai, Huazuo Gao, Yiyang Ma, Chengyue Wu, Bingxuan Wang, Zhenda Xie, Yu Wu, Kai Hu, Jiawei Wang, Yaofeng Sun, Yukun Li, Yishi Piao, Kang Guan, Aixin Liu, Xin Xie, Yuxiang You, Kai Dong, Xingkai Yu, Haowei Zhang, Liang Zhao, Yisong Wang, and Chong Ruan. Deepseek-vl2: Mixture-of-experts vision-language models for advanced multimodal understanding, 2024. arXiv:abs/2412.10302.
- [7] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. Deepseek-coder: When the large language model meets programming – the rise of code intelligence, 2024. arXiv:abs/2401.14196.
- [8] DeepSeek-AI, Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y. Wu, Yukun Li, Huazuo Gao, Shirong Ma, Wangding Zeng, Xiao Bi, Zihui Gu, Hanwei Xu, Damai Dai, Kai Dong, Liyue Zhang, Yishi Piao, Zhibin Gou, Zhenda Xie, Zhewen Hao, Bingxuan Wang, Junxiao Song, Deli Chen, Xin Xie, Kang Guan, Yuxiang You, Aixin Liu, Qiusi Du, Wenjun Gao, Xuan Lu, Qinyu Chen, Yaohui Wang, Chengqi Deng, Jiashi Li, Chenggang Zhao, Chong Ruan, Fuli Luo, and Wenfeng Liang. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence, 2024. arXiv:abs/2406.11931.
- [9] DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Yang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jin Chen, Jingyang Yuan, Junjie Qiu, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Liyue Zhang, Meng Li, Miaojuan Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qihao Zhu, Qinyu Chen, Qiusi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruizhe Pan, Runxin Xu, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Size Zheng, T. Wang, Tian Pei, Tian Yuan, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanbiao Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaojin Shen, Xiaokang Chen, Xiaoshan Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. Deepseek-v3 technical report, 2025. arXiv:abs/2412.19437.
- [11] DeepSeek-AI, :, Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiusi Du, Zhe Fu, Huazuo Gao, Kaige Gao, Wenjun Gao, Ruiqi Ge, Kang Guan, Daya Guo, Jianzhong Guo, Guangbo Hao, Zhewen Hao, Ying He, Wenjie Hu, Panpan Huang, Erhang Li, Guowei Li, Jiashi Li, Yao Li, Y. K. Li, Wenfeng Liang, Fangyun Lin, A. X. Liu, Bo Liu, Wen Liu, Xiaodong Liu, Xin Liu, Yiyuan Liu, Haoyu Lu, Shanghao Lu, Fuli Luo, Shirong Ma, Xiaotao Nie, Tian Pei, Yishi Piao, Junjie Qiu, Hui Qu, Tongzheng Ren, Zehui Ren, Chong Ruan, Zhangli Sha, Zhihong Shao, Junxiao Song, Xuecheng Su, Jingxiang Sun, Yaofeng Sun, Minghui Tang, Bingxuan Wang, Peiyi Wang, Shiyu Wang, Yaohui Wang, Yongji Wang, Tong Wu, Y. Wu, Xin Xie, Zhenda Xie, Ziwei Xie, Yiliang Xiong, Hanwei Xu, R. X. Xu, Yanhong Xu, Dejian Yang, Yuxiang You, Shuiping Yu, Xingkai Yu, B. Zhang, Haowei Zhang, Lecong Zhang, Liyue Zhang, Mingchuan Zhang, Minghua Zhang, Wentao Zhang, Yichao Zhang, Chenggang Zhao, Yao Zhao, Shangyan Zhou, Shunfeng Zhou, Qihao Zhu, and Yuheng Zou. Deepseek llm: Scaling open-source language

- models with longtermism, 2024. arXiv:abs/2401.02954.
- [12] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiusi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shutong Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. arXiv:abs/2501.12948.
 - [13] Ayman Odeh, Nada Odeh, and Abdul Salam Mohammed. A comparative review of AI techniques for automated code generation in software development: advancements, challenges, and future directions. *TEM Journal*, 13(1):726, 2024.
 - [14] Muhammad Fawad Akbar Khan, Max Ramsdell, Erik Falor, and Hamid Karimi. Assessing the promise and pitfalls of chatgpt for automated code generation, 2023. arXiv:abs/2311.02640.
 - [15] Zhijie Liu, Yutian Tang, Xiapu Luo, Yuming Zhou, and Liang Feng Zhang. No need to lift a finger anymore? assessing the quality of code generation by chatgpt. *IEEE Transactions on Software Engineering*, 50(6):1548–1584, 2024. DOI:10.1109/TSE.2024.3392499.
 - [16] Mohammed Latif Siddiq, Lindsay Roney, Jiahao Zhang, and Joanna C. S. Santos. Quality assessment of chatgpt generated code and their use by developers. In *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*, pages 152–156, 2024.
 - [17] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. An empirical study of the non-determinism of chatgpt in code generation. *ACM Transactions on Software Engineering and Methodology*, 2024. DOI:10.1145/3697010.
 - [18] Dapeng Yan, Zhipeng Gao, and Zhiming Liu. A closer look at different difficulty levels code generation abilities of chatgpt. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1887–1898, 2023. DOI:10.1109/ASE56229.2023.00096.
 - [19] Carlos Eduardo Andino Coelho, Mohammed Nazeh Alimam, and Rand Kouatly. Effectiveness of chatgpt in coding: A comparative analysis of popular large language models. *Digital*, 4:114–125, 2024.
 - [20] Yue Liu, Thanh Le-Cong, Ratnadira Widyasari, Chakkrit Tantithamthorn, Li Li, Xuan-Bach D. Le, and David Lo. Refining ChatGPT-generated code: Characterizing and mitigating code quality issues. *ACM Trans. Softw. Eng. Methodol.*, 33(5), June 2024. DOI:10.1145/3643674.
 - [21] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23*, Red Hook, NY, USA, 2024. Curran Associates Inc.
 - [22] Yunhe Feng, Sreecharan Vanam, Manasa Cherukupally, Weijian Zheng, Meikang Qiu, and Haihua Chen. Investigating code generation performance of chatgpt with crowdsourcing social data. In *2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)*, pages 876–885, 2023. DOI:10.1109/COMPSAC57700.2023.00117.
 - [23] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebggen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. arXiv:abs/2107.03374.
 - [24] Jialu Zhang, José Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. Repairing bugs in python assignments using large language models, 2022. arXiv:abs/2209.14876.
 - [25] Jacob Doughty, Zipiao Wan, Anishka Bompelli, Jubahed Qayum, Taozhi Wang, Juran Zhang, Yujia Zheng, Aidan Doyle, Pragnya Sridhar, Arav Agarwal, Christopher Bogart, Eric Keylor, Can Kultur, Jaromir Savelka, and Majd Sakr. A comparative study of AI-generated (GPT-4) and human-crafted MCQs in programming education. In *Proceedings of the 26th Australasian Computing Education Conference, ACE '24*, page 114–123, New York, NY, USA, 2024. Association for Computing Machinery. DOI:10.1145/3636243.3636256.
 - [26] Arto Hellas, Juho Leinonen, Sami Sarsa, Charles Koutchme, Lilja Ku-janpää Juha, and Sorva. Exploring the responses of large language models to beginner programmers' help requests. In *Proceedings of the 2023 ACM Conference on International Computing Education Research - Volume 1, ICER '23*, page 93–105, New York, NY, USA, 2023. Association for Computing Machinery. DOI:10.1145/3568813.3600139.
 - [27] Manh Hung Nguyen, Sebastian Tschitschek, and Adish Kumar Singla. Large language models for in-context student modeling: Synthesizing student's behavior in visual programming from one-shot observation, 2023. arXiv:abs/2310.10690.
 - [28] Natalie Kiesler and Daniel Schiffner. Large language models in introductory programming education: ChatGPT's performance and implications for assessments. *ArXiv*, abs/2308.08572, 2023.
 - [29] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*, New York, NY, USA, 2024. Association for Computing Machinery. DOI:10.1145/3597503.3623343.
 - [30] Zhe Liu, Chunyang Chen, Junjie Wang, Xing Che, Yuekai Huang, Jun Hu, and Qing Wang. Fill in the blank: Context-aware automated text input generation for mobile gui testing. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 1355–1367, 2023. DOI:10.1109/ICSE48619.2023.00119.
 - [31] Toufique Ahmed and Premkumar Devanbu. Few-shot training llms for project-specific code-summarization. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, ASE '22*, New York, NY, USA, 2023. Association for Computing Machinery. DOI:10.1145/3551349.3559555.
 - [32] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. CodeGen: An open large language model for code with multi-turn program synthesis. In *The Eleventh International Conference on Learning Representations*, 2023.
 - [33] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. InCoder: A generative model for code infilling and synthesis. In *The Eleventh International Conference on Learning Representations*, 2023.
 - [34] Frank F. Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine*

- Programming*, MAPS 2022, page 1–10, New York, NY, USA, 2022. Association for Computing Machinery. DOI:[10.1145/3520312.3534862](https://doi.org/10.1145/3520312.3534862).
- [35] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. Training a helpful and harmless assistant with reinforcement learning from human feedback, 2022. arXiv:[abs/2204.05862](https://arxiv.org/abs/2204.05862).
 - [36] Stephen Casper, Xander Davies, Claudia Shi, Thomas Krendl Gilbert, Jérémy Scheurer, Javier Rando, Rachel Freedman, Tomasz Korbak, David Lindner, Pedro Freire, Tony Wang, Samuel Marks, Charbel-Raphaël Segerie, Micah Carroll, Andi Peng, Phillip Christoffersen, Mehul Damani, Stewart Slocum, Usman Anwar, Anand Siththaranjan, Max Nadeau, Eric J. Michaud, Jacob Pfau, Dmitrii Krashennnikov, Xin Chen, Lauro Langosco, Peter Hase, Erdem Bıyık, Anca Dragan, David Krueger, Dorsa Sadigh, and Dylan Hadfield-Menell. Open problems and fundamental limitations of reinforcement learning from human feedback, 2023. arXiv:[abs/2307.15217](https://arxiv.org/abs/2307.15217).
 - [37] Nat McAleese, Rai Pokorny, Juan Uribe, Evgenia Nitishinskaya, Maja Trębacz, and Jan Leike. Llm critics help catch llm bugs, 06 2024. arXiv:[abs/2407.00215](https://arxiv.org/abs/2407.00215).
 - [38] Colin White, Samuel Dooley, Manley Roberts, and Arka Pal. Livebench: A challenging, contamination-free LLM benchmark, 2024. arXiv:[abs/2406.19314](https://arxiv.org/abs/2406.19314).
 - [39] Barbara Kitchenham and Stuart Charters. Guidelines for performing systematic literature reviews in software engineering. Technical Report EBSE 2007-001, Keele University and Durham University Joint Report, 2007.
 - [40] Kai Petersen, Sairam Vakkalanka, and Ludwik Kuzniarz. Guidelines for conducting systematic mapping studies in software engineering: An update. *Information and Software Technology*, 64:1–18, 2015. DOI:[10.1016/j.infsof.2015.03.007](https://doi.org/10.1016/j.infsof.2015.03.007).
 - [41] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. Program synthesis with large language models. *ArXiv*, abs/2108.07732, 2021.
 - [42] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921, Brussels, Belgium, 2018. Association for Computational Linguistics. DOI:[10.18653/v1/D18-1425](https://doi.org/10.18653/v1/D18-1425).
 - [43] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. CodeGeeX: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD '23*, page 5673–5684, New York, NY, USA, 2023. Association for Computing Machinery. DOI:[10.1145/3580305.3599790](https://doi.org/10.1145/3580305.3599790).
 - [44] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022. DOI:[10.1126/science.abq1158](https://doi.org/10.1126/science.abq1158).
 - [45] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q. Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. MultiPL-E: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, 49(7):3675–3691, 2023. DOI:[10.1109/TSE.2023.3267446](https://doi.org/10.1109/TSE.2023.3267446).
 - [46] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.
 - [47] Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D. Man-
ning, and Chelsea Finn. DetectGPT: Zero-shot machine-generated text detection using probability curvature, 2023. arXiv:[abs/2301.11305](https://arxiv.org/abs/2301.11305).
 - [48] Hugo Larochelle, Dumitru Erhan, and Yoshua Bengio. Zero-data learning of new tasks. In *Proceedings of the 23rd National Conference on Artificial Intelligence - Volume 2, AAAI’08*, page 646–651. AAAI Press, 2008.
 - [49] Christoph H. Lampert, Hannes Nickisch, and Stefan Harmeling. Attribute-based classification for zero-shot visual object categorization. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 36(3):453–465, 2014. DOI:[10.1109/TPAMI.2013.140](https://doi.org/10.1109/TPAMI.2013.140).
 - [50] Yuling Shi, Hongyu Zhang, Chengcheng Wan, and Xiaodong Gu. Between lines of code: Unraveling the distinct patterns of machine and human programmers, 2024. arXiv:[abs/2401.06461](https://arxiv.org/abs/2401.06461).
 - [51] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. Automatic generation of programming exercises and code explanations using large language models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research - Volume 1, ICER ’22*, page 27–43, New York, NY, USA, 2022. Association for Computing Machinery. DOI:[10.1145/3501385.3543957](https://doi.org/10.1145/3501385.3543957).
 - [52] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. Exploring the potential of chatgpt in automated code refinement: An empirical study. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE ’24*, New York, NY, USA, 2024. Association for Computing Machinery. DOI:[10.1145/3597503.3623306](https://doi.org/10.1145/3597503.3623306).
 - [53] Zhiyu Li, Shuai Lu, Daya Guo, Nan Duan, Shailesh Jannu, Grant Jenks, Deep Majumder, Jared Green, Alexey Svyatkovskiy, Shengyu Fu, and Neel Sundaresan. Automating code review activities by large-scale pre-training, 2022. arXiv:[abs/2203.09095](https://arxiv.org/abs/2203.09095).
 - [54] Tao Xiao, Christoph Treude, Hideaki Hata, and Kenichi Matsumoto. Devgpt: Studying developer-chatgpt conversations. In *In Proceedings of the International Conference on Mining Software Repositories (MSR 2024)*, 2024.
 - [55] Bandit. <https://bandit.readthedocs.io/>, 2023. [Online; accessed January 2025].
 - [56] GitHub Inc. Use of a broken or weak cryptographic hashing algorithm on sensitive data. <https://codeql.github.com/codeql-query-help/python/py-weak-sensitive-data-hashing/>, 2022. [Online; accessed January 2025].
 - [57] Pylint. <https://pylint.pycqa.org/>, 2023. [Online; accessed January 2025].